01 / VISÃO GERAL

Resumo do trabalho e plano de execução

Ride-pooling multiobjetivo
Emissão de CO_2 × tempo perdido pelo passageiro

Bruno Prado Santos · Eduardo Almeida Queiroz · Lucas Cerqueira Portela
CEFET-MG · Otimização Multiobjetivo · 15 de outubro de 2026

O que o trabalho pretende responder

Quanto a frota pode economizar em deslocamento e emissão quando as viagens são compartilhadas, e quanto tempo adicional isso custa aos usuários? A resposta será uma fronteira de Pareto: um conjunto de escolhas eficientes entre os dois objetivos.

Situação real do repositório

A modelagem, a leitura das instâncias, a avaliação de soluções e uma referência exata pequena já existem. **Não há implementação de NSGA-II, MOILS ou ILS de roteamento no código atual.** O procedimento BORDO já escrito é uma heurística de horários para uma rota fixa; ele não substitui a metaheurística que deverá escolher as rotas.

As baterias das fases 1 e 2 foram executadas nesta revisão e terminaram sem falhas. Isso reproduz os exemplos existentes, mas não comprova a correção para qualquer entrada. Foram identificadas lacunas relevantes no validador e divergências entre texto, comentários e execução.

Encaminhamento proposto

• **Primeiro:** consolidar as regras do modelo e reforçar o validador independente.

• **Depois:** implementar uma MOILS própria, aproveitando as estruturas e o avaliador existentes. Se a comparação dos três métodos continuar no escopo, acrescentar NSGA-II próprio e ε-restrito + ILS.

• **Por fim:** conferir instâncias pequenas contra o gabarito, executar repetições controladas nas maiores e produzir tabelas de qualidade, viabilidade e tempo.

Como ler este documento

Modelo e dados (p. 2); código existente (p. 3); resultados reproduzidos (p. 4); correções prioritárias (p. 5); implementação própria (p. 6-7); testes de correção (p. 8); validação experimental (p. 9-10); cronograma (p. 11); reprodução e fontes (p. 12).

Escopo desta entrega: resumo técnico e planejamento. Os algoritmos e as correções propostos ainda não foram implementados. Os programas e os resultados originais foram preservados; a reexecução da fase 2 gravou cópias em tmp/pdfs/validacao/.

02 / MODELO

O problema, os dados e as hipóteses

O projeto resolve um **Dial-a-Ride Problem (DARP)**: cada solicitação possui coleta, entrega e quantidade de passageiros. Os veículos começam e terminam no depósito. A ordem das visitas e os horários devem respeitar o serviço contratado. [L1-L3]

Dois objetivos de minimização

f1 = φ × distância total. Com φ constante por cenário, minimizar emissão equivale a minimizar distância.

f2 = Σ q_i [max(0, R_i - $\bar{t}_i$) + max(0, B_i - ρ_i)].
R é o tempo a bordo, $\bar{t}$ é o percurso direto, B é o início do embarque e ρ é o horário desejado. O resultado é medido em passageiro-minuto. Esta é a expressão efetivamente usada no avaliador.

A coleta deve ocorrer antes da entrega, no mesmo veículo; cada nó deve ser atendido uma vez. Também se impõem capacidade, janelas de tempo, limite de tempo a bordo, limite de duração e disponibilidade de frota. A emissão é de toda a rota, incluindo deslocamentos vazios de saída, retorno e reposicionamento.

Dados disponíveis

Coleção	Arquivos locais	Uso atual
bnc	a2-16, a2-24, a4-48, a8-96; b2-16, b4-48, b8-96	7 instâncias; 16 a 96 solicitações; depósito final já explícito.
tabu	pr01, pr02, pr07, pr11; pr01.res e pr07.res	4 instâncias; 24 a 48 solicitações; depósito final é duplicado pelo leitor.

Os benchmarks usam distância euclidiana e velocidade unitária: numericamente, tempo de percurso = distância. **Não interpretar essas unidades automaticamente como quilômetros ou como emissões reais em gramas.** É necessário definir a conversão antes de aplicar fatores físicos de emissão.

Experimentos que ainda precisam ser fechados

O relatório propõe carros de capacidade 4 e vans de 8 a 15 lugares, com limites proporcionais de tempo a bordo. Os resultados atuais ainda usam os parâmetros dos benchmarks, φ = 1 e α = None (limite global do arquivo). A comparação final entre os dois cenários ainda não foi executada.

Capacidade 4 não impõe atendimento individual: um carro também pode transportar solicitações distintas ao mesmo tempo. Se o cenário individual proibir agrupamento, exigir explicitamente no máximo uma solicitação ativa por vez. Fixar também a regra de frota: mesma disponibilidade ou políticas de dimensionamento claramente descritas.

Sem desvios e sem espera é possível obter f2 = 0, quando as janelas e a frota permitem. A afirmação de que isso impede visitas intermediárias depende de tempos de serviço intermediários positivos e da desigualdade triangular. Não implica um veículo exclusivo para cada solicitação.

03 / MAPA DO CÓDIGO

O que já foi construído

Componente	Função implementada	Situação
instancia.py	Leitura dos dois formatos, distâncias, tempos, classificação ida/volta, inferência de ρ e redução de instâncias.	Implementado
solucao.py	Rotas, cópias, cobertura dos nós, pareamento, precedência e construção sem agrupamento.	Implementado
avaliador.py	Horários CEDO, BORDO e OTIMO; objetivos, carga e verificações de viabilidade; agenda externa.	Implementado; reforçar checagens
leitor_res.py	Leitura de soluções publicadas com campos rotulados ou sem rótulos.	Implementado
pareto.py	Dominância, filtragem, soma de frentes, normalização, hipervolume 2D e cobertura.	Implementado; revisar convenções
exato.py	Enumeração de ordens, frentes por subconjunto e combinação por programação dinâmica.	Gabarito pequeno
mip.py	Formulação Gurobi e varredura ε-restrita; exportação de rotas e horários.	Referência; reforçar certificado
experimentos/	validar_fase1.py e fase2_fronteira.py; exportação de CSV e gráfico.	Baterias existentes
Metaheurísticas	NSGA-II, MOILS e ε-restrito + ILS para escolher sequências/agrupamentos.	Ainda pendentes

Por que o agendamento ganhou três políticas

CEDO agenda cada atendimento no primeiro instante possível. Pode embarcar cedo demais e manter a pessoa esperando no veículo até a abertura da janela de entrega. **BORDO** usa folgas para atrasar embarques e reduzir tempo a bordo. **OTIMO** usa programação linear via SciPy para minimizar f2 da sequência fixa. [L3]

No código, **OTIMO é o padrão**. O diário de desenvolvimento prevê BORDO como padrão futuro das metaheurísticas. Essa escolha deve ficar explícita na configuração de cada execução, pois muda os objetivos e até quais rotas são consideradas viáveis.

Dependências atuais

pyproject.toml declara NumPy, SciPy e Matplotlib; Gurobi é opcional. Não há dependência de pymoo instalada por esse arquivo. A menção a pymoo está no planejamento textual e precisa ser substituída se a exigência for implementar a metaheurística do zero. [L1-L2]

04 / EVIDÊNCIA ATUAL

Resultados que foram reproduzidos

As duas baterias existentes terminaram com “TODAS AS CONFERENCIAS PASSARAM”. A primeira leu as 11 instâncias e reproduziu distância, carga, espera e tempo a bordo de pr01.res e pr07.res dentro das tolerâncias declaradas. [L4]

Caso / procedimento	f1 (φ = 1)	f2	Observação
pr01, horários publicados	190,02	1.233,25	3 veículos; sem violações observadas.
pr01, mesmas rotas + OTIMO	190,02	1.183,64	Melhoria apenas nos horários.
pr01, mesmas rotas + CEDO	190,02	1.700,33	10 violações; objetivo não é solução viável.
pr07, horários publicados	291,71	1.716,86	4 veículos; sem violações observadas.
pr07, mesmas rotas + OTIMO	291,71	1.580,63	Mesmo deslocamento; serviço melhor.

Gabarito: pr01 reduzida a cinco solicitações

Foram mantidos os pedidos originais **[1, 2, 3, 4, 5]**, com m = 5, Q = 6 e limite global a bordo de 90. A execução mostrou que **todos os cinco são de volta** (janela na entrega), apesar do comentário do script dizer “2 de volta e 3 de ida”. Enumeração + OTIMO e Gurobi retornaram os mesmos quatro pontos; diferença máxima de aproximadamente $1{,}28 \times 10^{-13}$. [L5]

Ponto	f1	f2
1	47,0253	89,4142
2	47,6721	70,2116
3	49,1145	39,1136
4	49,9657	0,0000

Com BORDO, a enumeração retornou dez pontos; todos foram estritamente dominados pela frente com OTIMO. O hipervolume normalizado, usando a referência comum do script, foi **0,9222 para OTIMO** e **0,5738 para BORDO**. Mais pontos, portanto, não significaram melhor qualidade.

O que estes resultados demonstram

Há coerência entre duas implementações para esse recorte pequeno. Isso valida uma referência inicial, mas não prova que o modelo, o avaliador ou o método exato estejam corretos em todas as entradas. Também não valida ainda os cenários de carro e van propostos no relatório.

05 / PRIORIDADES DE VALIDAÇÃO

O que corrigir antes de confiar na busca

1. Agenda externa pode ser aceita mesmo sendo impossível

Foi executado um caso sintético com uma solicitação: depósito (0,0), coleta (1,0), entrega (2,0), janela de coleta [10,90], serviço zero e percurso direto 1. Ao fornecer $B_{coleta} = B_{entrega} = 0$ e partida 0, avaliar_com_B retornou **viavel=True**, sem violações, com f2 = 0. A agenda viola a janela inferior, o deslocamento e o tempo mínimo a bordo. [L3; prova executada nesta revisão]

Ação: criar um verificador independente e adicionar limites inferiores e superiores das janelas, encadeamento de todos os trechos, janelas do depósito, carga não negativa/final zero e R ≥ t̄. Não usar max(0, ...) para esconder uma violação física. Primeiro validar a agenda, depois computar a qualidade.

2. “Serviço perfeito” não é um extremo eficiente certificado

O CSV registra (376,92; 0) usando 24 veículos. A própria construção com **6 veículos** produziu uma solução viável com **(341,9457; aproximadamente 0)**; logo a referência de 24 veículos é dominada quando ambas podem ser usadas no mesmo cenário com m ≥ 24. O aumento de 98% de distância é apenas comparação entre duas construções, não o custo mínimo comprovado de zerar f2. [L4-L5]

Ação: chamar esses pontos de soluções de referência. Para obter extremo, minimizar f2 e depois f1 em desempate lexicográfico, mantendo os mesmos parâmetros. Da mesma forma, a distância 190,02 é referência publicada, não um ótimo certificado aqui para todo cenário com frota ampliada.

3. “OTIMO” pode mudar silenciosamente de método

Sem SciPy, _agendar_otimo recorre a BORDO. Falha do programa linear também aciona BORDO. **Ação:** registrar a política realmente executada e o status do PL; em testes exatos, falhar explicitamente se o solver estiver indisponível. Distinguir inviabilidade comprovada de falha numérica. [L3]

4. Certificado exato e convenções precisam ser explícitos

fronteira_epsilon aceita uma solução quando há incumbente, sem exigir status ótimo; o limite de tempo, o limite de pontos e o passo fixo podem interromper a referência. O peso $f1 + 10^{-4} f2$ não garante ordem lexicográfica para qualquer escala. **Ação:** registrar status, gap, limites e motivo da parada, e usar desempate hierárquico; confrontar a frente com enumeração nos casos pequenos. [L6]

A cobertura atual usa dominância estrita; o relatório fala em dominância fraca. Definir qual será usada e testar conjuntos iguais. Alinhar ainda a definição de ρ, a inclusão de idas/voltas e α: o relatório seleciona idas e usa ρ = e; o código inclui voltas com ρ inferido e aplica max(0, B - ρ). [L1-L3]

06 / ESCOPO E ARQUITETURA

Como implementar a metaheurística à mão

Interpretação de trabalho para “à mão”

Escrever os operadores, a seleção, a busca, a perturbação e o arquivo de Pareto em código próprio, sem chamar um pacote que entregue a metaheurística completa. Esta é uma hipótese de planejamento, pois a regra da disciplina ainda foi descrita como aparente. Não é necessário reescrever leitura de dados ou rotinas gráficas.

Uso	Proposta
Implementação própria obrigatória	MOILS; se mantidos no estudo, NSGA-II e ILS; inserção/remoção de pares; critérios de aceitação; controle de diversidade e parada.
Apoio geral	Biblioteca padrão, random.Random(seed), math, csv, json, time e testes; NumPy/Matplotlib para cálculo e apresentação, se permitidos.
Referência de validação	SciPy/linprog e Gurobi em execução separada para conferir horários e instâncias pequenas; guardar seus certificados.
Se a regra excluir qualquer solver na busca	Executar a busca com BORDO explicitamente e avaliar a perda frente a OTIMO fora da busca. Não apresentar esse resultado como otimização exata dos horários.

Sequência recomendada

Começar pela MOILS é uma escolha de engenharia: ela reutiliza diretamente as listas de rotas e permite testar cada movimento antes de montar um algoritmo populacional. Isso reduz a quantidade de componentes novos para a primeira entrega funcional. O compromisso atual do relatório com três métodos deve ser revisto se houver prazo apenas para um.

Núcleo compartilhado proposto

• **operadores.py:** retirar e reinserir coleta/entrega juntas; trocar solicitações entre rotas; abrir/fechar rota; perturbar sem perder pedidos.

• **construtivo.py:** inserir pares em posições p < d, avaliar todas as rotas elegíveis e permitir nova rota apenas se houver veículo disponível.

• **moils.py, nsga2.py, epsilon_ils.py:** ciclos próprios, parâmetros e contador de avaliações.

• **validacao.py e tests/:** checagem independente, casos calculáveis à mão e comparação com referências.

Um 2-opt convencional ou a inversão livre de nós pode trocar entrega e coleta de posição. Começar com movimentos de pares; qualquer operador posterior deve passar pelas mesmas checagens. Se uma inserção falhar, não apagar a solicitação nem ampliar a frota silenciosamente.

07 / PASSOS DOS ALGORITMOS

MOILS própria e extensões previstas

MOILS: primeira implementação

Usar múltiplas soluções iniciais: ordem por ρ, atendimento sem agrupamento e inserções aleatórias viáveis. Em cada passo, selecionar uma solução do arquivo, aplicar busca local e guardar vizinhos viáveis não dominados. Remover pontos dominados e duplicatas. Perturbar para sair do ótimo local e reiniciar após estagnação. Esta é uma variante proposta para o projeto, que deverá ser descrita no relatório.

```
rng = Random(seed)
A = filtrar_viaveis(construir_iniciais(rng))
while avaliacoes < limite and tempo < tempo_max:
    s = selecionar_regiao_pouco_explorada(A, rng)
    s = perturbar_pares(copiar(s), rng)
    s = reparar_ou_reconstruir(s, rng)
    if s is None: continue
    # Explorar movimentos; inserir todo vizinho viavel em A.
    busca_local_pareto(s, A, avaliar, rng, limite)
    A = filtrar_e_remover_duplicatas(A)
return A
```

Na busca local, aceitar dominantes e explorar alternativas incomparáveis; descartar dominadas. Para conter custo, limitar o número de expansões por solução e selecionar regiões por densidade. Guardar histórico de soluções viáveis para uma auditoria final. Se limitar o arquivo, registrar a regra de descarte e separar essa aproximação do filtro de Pareto.

NSGA-II próprio, se mantido no escopo

Implementar população, ordenação em frentes, crowding distance, torneio, cruzamento, mutação e seleção elitista de pais + filhos. A primeira versão pode usar comparação $O(N^2)$ para frentes e preservar extremos no cálculo de diversidade. Esses componentes são os mecanismos centrais descritos por Deb et al. [E1]

Para o DARP, propor cromossomo como permutação de solicitações com parâmetros de inserção; usar cruzamento de ordem e mutação por troca/realocação. O decodificador insere o par com precedência, verifica capacidade e agenda, e rejeita ou repara inserções inviáveis. Testar sua capacidade de produzir rotas diversas: um decodificador muito restritivo pode excluir bons agrupamentos.

ε-restrito + ILS: terceiro método

Escolher uma grade de ε; executar ILS próprio minimizando f1 sujeito a f2 ≤ ε. Compartilhar operadores e usar as soluções do ε anterior como partida, respeitando orçamento total igual ao dos outros métodos. Reportar esse procedimento como **aproximação heurística**; ele não é o método MIP exato já existente.

A busca sempre devolve soluções viáveis. Se não existir inicial viável, retornar estado explícito de falha construtiva; isso não prova que a instância seja inviável.

08 / TESTES POR CAMADA

Como validar a correção dos códigos

Antes de avaliar “qual algoritmo é melhor”, provar que entradas, rotas, horários e métricas significam o que o modelo afirma. Os testes abaixo são entregas planejadas; somente as baterias e as sondagens da p. 4-5 já foram executadas.

Camada	Casos necessários	Critério de aceite
Leitura e redução	Dois formatos; depósito duplicado; pedidos repetidos em reduzir; IDs ausentes; cabeçalho inválido; conservação de pares.	Contagem, IDs, demandas e janelas corretos; entrada inválida rejeitada com mensagem clara.
Estrutura e operadores	Coleta ausente; nó duplicado/inválido; par separado; entrega antes da coleta; solução vazia; cópias independentes.	Cada pedido uma vez, no mesmo veículo; nenhuma mutação altera a solução original.
Viabilidade completa	Excesso e carga negativa; janela inferior/superior; trecho sem tempo; R abaixo do direto/acima do limite; duração e depósito.	Verificador rejeita cada violação isolada e aceita casos de fronteira dentro da tolerância.
Agendamento	Rota fixa com CEDO, BORDO e PL; caso de janela na entrega; dependência ausente; status do PL.	f1 constante; OTIMO ≤ BORDO em f2 quando ambos viáveis e PL certificado.
Pareto e métricas	Dominância, igualdade, duplicatas, conjunto vazio, eixo constante, referência HV inválida.	Concordar com cálculos manuais e um filtro independente por pares.
Algoritmos	Mesmo seed; orçamento exato; arquivo viável; diversidade; elitismo; nenhuma perda de pedido.	Reprodução determinística por avaliações; parada respeitada; 100% das saídas válidas.

Três testes com resposta calculável à mão

Rota de uma pessoa: depósito (0,0), coleta (1,0), entrega (3,0), serviço zero, q = 2, Q = 2, janelas [0,100] e ρ = 1. Com partida 0, B = (1,3) e retorno 6: distância 6, R = 2, f1 = 6φ e f2 = 0. Se ambos B atrasarem 4, sem mudar a partida e com duração permitida, a espera ponderada sobe para 8.

Pareto: em {(1,4), (2,2), (4,1), (3,3), (2,2)}, a frente é {(1,4), (2,2), (4,1)}. Com referência (5,5), o HV bruto é 11; adicionar um ponto dominado não o altera. Com cobertura fraca, C(A,A) = 1; com a estrita atual, C(A,A) = 0 para uma frente sem duplicatas.

NSGA-II: para {(0,1), (0,5;0,5), (1,0)}, os extremos recebem distância infinita e o centro recebe 2, somando as contribuições normalizadas dos dois objetivos. Tratar amplitude zero sem divisão por zero.

09 / VALIDAÇÃO EM ESCALA CRESCENTE

Do gabarito à qualidade da metaheurística

Etapa A - instâncias minúsculas e independência

Criar casos artificiais de 1 a 3 pedidos com valores manuais; enumerar todas as sequências e alocações possíveis, com e sem agrupamento. Comparar resultados com a implementação principal usando um verificador que recompute distâncias e restrições sem reutilizar a mesma rotina de viabilidade. Isso evita que duas etapas compartilhem o mesmo erro.

Etapa B - recortes reais de cinco pedidos

Manter pr01-r5 atual como regressão. Acrescentar um recorte só de ida e outro misto, por exemplo [13,14,15,16,17] e [1,2,13,14,15] em pr01, confirmando a classificação na execução. Usar exatamente os mesmos m, Q, φ, α, janelas e política na referência e na metaheurística.

Para o modo com OTIMO, comparar a fronteira da enumeração com o MIP certificado e revalidar as agendas. Para o modo sem solver, comparar também com a enumeração sob BORDO: ela é referência **daquela política**. Em separado, medir a distância para a referência com horários ótimos.

Critérios para aprovar a primeira versão

100% das soluções devolvidas válidas; nenhuma solução domina estritamente o gabarito verdadeiro sob os mesmos parâmetros; arquivos internamente não dominados; orçamento e sementes registrados. Medir a fração dos quatro pontos recuperados e o HV relativo em cada execução. Recuperar todos os pontos é uma meta de qualidade, não uma propriedade garantida de uma metaheurística.

Se algum ponto parecer superar o gabarito, investigar primeiro parâmetros, política de horários, checagens e certificado exato. Não considerar automaticamente que a metaheurística melhorou um ótimo provado.

Etapa C - instâncias médias e grandes

Progredir de b2-16/a2-16 para pr01/pr07 e depois pr02, a4-48/b4-48 e a8-96/b8-96, conforme o tempo medido. Não prometer enumeração completa para oito pedidos: o número de ordens cresce rapidamente. Em escala maior, usar o conjunto não dominado da união das execuções como **referência empírica**, sem chamá-lo de ótimo.

Testes de propriedades e auditoria

• Permutar a ordem das rotas não altera os objetivos; incluir rota vazia não acrescenta distância.

• Dobrar φ dobra f1 e preserva f2; ampliar capacidade/frota expande o conjunto viável do modelo.

• Mover pares preserva cobertura; 10.000 movimentos aleatórios em instâncias pequenas devem manter os invariantes estruturais. A inviabilidade temporal de um candidato deve ser tratada pela política da busca.

Reavaliar as saídas finais com o mesmo verificador independente. Se reotimizar horários de todos os candidatos com PL, refiltrar a união: candidatos antes dominados podem reaparecer; registrar o custo desse pós-processamento.

10 / PROTOCOLO EXPERIMENTAL

Como comparar os resultados com rigor

Esta é uma proposta inicial de experimento, ajustável após um piloto. Fechar parâmetros e critérios antes da campanha final e manter os mesmos por instância/cenário entre os algoritmos.

Elemento	Protocolo proposto
Repetições	Piloto com 5 sementes; campanha com 30 execuções independentes por algoritmo, instância e cenário. Registrar a lista de sementes.
Orçamento	Critério principal: mesmo número de avaliações (piloto: 10.000 por execução). Medir tempo real e chamadas por rota/PL. Fazer uma comparação adicional com tempo igual se o custo variar muito.
Ajuste	Separar instâncias de ajuste das de avaliação. Grade pequena e custo de ajuste equivalente para todos. Não escolher a melhor semente como resultado representativo.
Baselines	Construtivo sem busca, inserção aleatória viável e busca local sem perturbação. Inicialização comum quando as representações permitirem.
Métricas	Taxa de viabilidade, HV, C(A,B) e C(B,A), cardinalidade sem duplicatas, recuperação do gabarito, tempo e avaliações.
Registros	algoritmo, instância, pedidos originais, cenário, seed, parâmetros, política efetiva, versões, commit, f1/f2, rotas, horários, violações, tempo e histórico.

Normalização e leitura das métricas

Aplicar os mesmos mínimos e máximos a todos os métodos de um mesmo caso. Nas pequenas, usar limites comuns derivados da referência e das saídas viáveis; nas grandes, congelar limites comuns calculados sobre o conjunto da campanha. Usar, por exemplo, referência HV (1,1; 1,1), isto é, **(1.1, 1.1)** após normalização de todos os pontos no intervalo [0,1]. Não normalizar cada frente isoladamente.

HV maior indica melhor compromisso entre extensão e proximidade. C(A,B) é direcional; igualdade precisa seguir a convenção declarada. Cardinalidade sozinha não mede qualidade. Nas pequenas, HV relativo = HV obtido / HV do gabarito, com exatamente os mesmos limites e referência. Uma referência empírica grande só mede qualidade em relação ao que foi encontrado. [E2]

Resumo estatístico e efeito do agendamento

Apresentar mediana e intervalo interquartil das 30 execuções, curvas de HV por avaliações e boxplots. Agregar por instância para evitar tratar sementes como novos problemas. Se houver comparação inferencial entre três métodos em várias instâncias, planejar Friedman e pós-testes pareados com correção de multiplicidade, além de tamanho de efeito; não concluir superioridade geral a partir de um único recorte.

Comparar primeiro algoritmos com **o mesmo agendador**. Depois fazer uma análise separada BORDO × OTIMO. Se todos forem reavaliados com PL no fim, aplicar o mesmo pós-processamento e contabilizar seu tempo. Assim, o ganho do roteamento não será confundido com o ganho dos horários.

11 / CRONOGRAMA SUGERIDO

Planejamento prático de execução

Estimativa de **quatro semanas de trabalho da equipe**, a ajustar à disponibilidade e ao prazo da disciplina. A campanha final só começa depois da aprovação das etapas de correção e regressão.

Período	Atividades e entregas	Condição para avançar
Semana 1 Modelo e confiança	Consolidar ρ, idas/voltas, α, φ, frota e cenário individual. Corrigir validador, fallback e rótulos. Criar testes manuais; atualizar texto e configurações.	Casos inviáveis rejeitados; regressões pr01/pr07 preservadas; regras registradas.
Semana 2 Primeira busca própria	Implementar inserção de pares, movimentos, inicialização e MOILS. Instrumentar contadores e sementes. Testar recortes de ida, volta e mistos.	Arquivo 100% viável; nenhuma perda de pedido; comparação com gabaritos produzida.
Semana 3 Comparadores e piloto	Se exigidos, implementar NSGA-II e ε-restrito + ILS. Executar baselines e piloto de parâmetros; medir custo por avaliação e PL.	Componentes validados; protocolo, orçamento e parâmetros congelados.
Semana 4 Campanha e redação	Executar repetições e cenários, salvar dados brutos, gerar métricas e gráficos. Auditar saídas; reescrever conclusões e limitações.	Execução reproduzível; tabelas por instância; referência exata e empírica distinguidas.

Divisão sugerida entre três integrantes

Frente 1: modelo, testes e verificador independente. **Frente 2:** operadores e metaheurística própria. **Frente 3:** protocolo, referências exatas, registros e gráficos. Fazer revisão cruzada: quem escreve o avaliador não deve ser a única pessoa a escrever seu gabarito.

Se o prazo for curto

Priorizar uma MOILS própria completa, duas famílias de instâncias, recortes exatos variados, baselines e repetições. Registrar a redução de escopo e ajustar a promessa do relatório; é melhor entregar um método validado do que manter três implementações incompletas.

Checklist da entrega acadêmica

• Código próprio identificável, pseudocódigo e operadores descritos; retirar a previsão de pymoo se incompatível com a regra.

• Modelo e cenário iguais nos códigos, nas tabelas e no texto; unidades e fatores de emissão justificados.

• Testes executáveis, gabaritos, sementes, configurações e saídas brutas; certificado do solver quando houver alegação de exatidão.

• Fronteiras e métricas com parâmetros comuns; resultados de busca separados dos efeitos do agendamento.

12 / RASTREABILIDADE

Como reproduzir e de onde veio o resumo

Comandos existentes - executar na raiz do projeto

```
python codigo/experimentos/validar_fase1.py
python codigo/experimentos/fase2_fronteira.py
```

O segundo comando normalmente sobrescreve os CSVs e o PNG em resultados/. Nesta revisão, a saída foi redirecionada para tmp/pdfs/validacao/ sem alterar o script. Nenhuma metaheurística foi executada, pois ainda não existe no repositório.

Ambiente da reexecução: commit 8aa05e7; Python 3.14.6; NumPy 2.5.0; SciPy 1.18.0; Matplotlib 3.11.0; Gurobi 13.0.2. O ambiente de Python empacotado usado para gerar este PDF é distinto do ambiente usado para os testes. Data da inspeção: 15/10/2026. Tempos medidos não foram usados como benchmark de desempenho.

Fontes locais examinadas

[L1] relatorio/main.tex; relatorio/resumo_professor.tex; relatorio/RELATORIO_DESENVOLVIMENTO.md. Formulação, cenários pretendidos, histórico e fases futuras. Quando o texto diverge da execução, este resumo indica a diferença.

[L2] pyproject.toml; codigo/darp/instancia.py; codigo/darp/solucao.py; codigo/darp/leitor_res.py. Dependências, dados, representação e leitura de resultados publicados.

[L3] codigo/darp/avaliador.py; codigo/darp/pareto.py. Políticas efetivas, cálculo de objetivos, viabilidade e métricas.

[L4] codigo/experimentos/validar_fase1.py; dados/tabu/pr01.res; dados/tabu/pr07.res; execução da bateria nesta revisão. Distâncias e agendas publicadas e comparação de políticas.

[L5] codigo/experimentos/fase2_fronteira.py; codigo/darp/exato.py; resultados/fase2_extremos_pr01.csv; resultados/fase2_fronteira_pr01r5.csv; reexecução com saída isolada. Referência de cinco pedidos e extremos construídos.

[L6] codigo/darp/mip.py. Formulação, objetivo aumentado, status, limites e varredura ε-restrita. A coincidência observada não substitui o registro formal de otimalidade de cada resolução.

Referências metodológicas consultadas

[E1] Deb, K.; Pratap, A.; Agarwal, S.; Meyarivan, T. (2002). *A Fast and Elitist Multiobjective Genetic Algorithm: NSGA-II.* IEEE Transactions on Evolutionary Computation, 6(2), 182-197. DOI: 10.1109/4235.996017. Lista de publicações do coautor.

[E2] Zitzler, E.; Deb, K.; Thiele, L. (2000). *Comparison of Multiobjective Evolutionary Algorithms: Empirical Results.* Evolutionary Computation, 8(2), 173-195. Artigo no repositório da ETH. Apoio à interpretação de comparação de conjuntos; o protocolo proposto neste documento é uma decisão de projeto.

O capítulo de Assis et al. (2013) e os trabalhos de Cordeau e Laporte são referências já indicadas nos relatórios do repositório. Este resumo não constitui uma reprodução ou uma nova validação integral dessas publicações.

Próxima ação concreta

Implementar primeiro o verificador completo e os testes que hoje conseguem passar com agenda inválida. Só depois usar a metaheurística para gerar milhares de candidatos. O gabarito pequeno já disponível permite avaliar a busca sem depender de impressão visual da fronteira.